Cours MERN Stack – Groupe Décalé – Séance 5

Connecter React au Serveur Express avec Axios & Redux Toolkit Simplifié

Dr. Abdelweheb GUEDDES – École Polytechnique de Sousse – 2026–2027

Programme de la Séance (100 % Pratique, Zéro Tunnel Théorique) :

1. **Heure 1 (Dès la minute 5) :** On allume le serveur Express et on affiche ses articles dans React avec **Axios** en réutilisant simplement le `useState` et le `useEffect` de la Séance 4 !
2. **Heure 2 :** Pour partager les articles avec le compteur de la `Navbar` sans passer de `props`, on crée un **Coffre-Fort Redux Toolkit ultra-simple** (`createSlice` de 20 lignes).
3. **Heure 3 :** On branche le formulaire d'ajout, la suppression et les 3 petits défis guidés.

1 Étape 1 (Minute 5) : Démarrer Express, CORS & Installer Axios

1. Le Principe en Une Image : React (Port 5173) Parle à Express (Port 5000)

En Séance 4, nos articles étaient stockés uniquement dans le navigateur (`localStorage`). Aujourd'hui, le client React demande les articles au **Serveur Express** grâce à la bibliothèque **Axios**.

Comme React (`5173`) et Express (`5000`) tournent sur deux ports différents, Express doit ouvrir sa porte en activant **`app.use(cors())`** :



FIGURE 1 – Le dialogue Client-Serveur : Axios traverse le portier CORS et reçoit le JSON dans `response.data`.

2. TP Immédiat 1 : Lancer le Serveur Express et Configurer Axios (5 minutes)

A. Côté Serveur (`server/`) : Vérifiez que ces 2 lignes CORS figurent bien en haut de `server/server.js`, puis lancez le serveur dans un premier terminal :

```
const cors = require('cors');
app.use(cors()); // Indispensable pour autoriser React (Port 5173) !
```

```
cd server
npm install
npm start
```

B. Côté Client React (`client/`) : Ouvrez un second terminal, installez `axios`, et créez le petit fichier `src/api/axiosClient.js` (6 lignes) pour ne pas recopier l'adresse du serveur partout :

```
cd client
npm install axios
```

➥ Fichier `src/api/axiosClient.js` – exécuter avec `npm run dev`

```
import axios from 'axios';

// Instance Axios unique pointant vers notre serveur Express (Port 5000)
const axiosClient = axios.create({
  baseURL: 'http://localhost:5000/api'
});

export default axiosClient;
```

Repère Simple : Pourquoi Axios est beaucoup plus simple que `fetch()`. Avec `fetch()`, il fallait toujours écrire deux étapes (`fetch` puis `await res.json()`) et convertir en texte (`JSON.stringify`). Avec **Axios**, tout est automatique : les données du serveur arrivent directement prêtes à l'emploi dans **`response.data`** !

2 Étape 2 (Minute 15) : Première Victoire – Afficher et Ajouter des Articles avec Axios + `useState` (Comme en Séance 4 !)

Avant même de parler de Redux, voyons comme il est facile d'utiliser `axiosClient` avec les deux outils que vous connaissez déjà depuis la Séance 4 : **`useState`** et **`useEffect`**.

1. Les 3 Opérations Axios à Connaître par Cœur

Toutes vos communications avec le serveur Express se résument à ces 3 lignes très simples :

Action voulue	Ligne Axios à écrire	Ce que renvoie le serveur
1. Lire les articles	`const res = await axiosClient.get('/articles');`	`res.data` contient le tableau des articles.
2. Créer un article	`const res = await axiosClient.post('/articles', obj);`	`res.data` contient le nouvel article créé avec son `id`.
3. Supprimer un article	`await axiosClient.delete(`/articles/${id}`);`	L'article est supprimé sur le serveur.

TABLE 1 – Les 3 seules commandes Axios dont vous avez besoin aujourd'hui.

2. Exemple Concret : Charger les Articles dans un `useEffect`

Regardez comment on charge les articles du serveur à l'ouverture de la page avec un simple `useState` :

```
import React, { useState, useEffect } from 'react';
import axiosClient from '../api/axiosClient';

export default function TestAxiosSimple() {
  const [articles, setArticles] = useState([]);       // Liste des articles
  const [loading, setLoading] = useState(false);      // true pendant le chargement
  const [error, setError] = useState(null);           // Message si le serveur est eteint

  // Fonction qui interroge le serveur Express
  const chargerArticles = async () => {
    try {
      setLoading(true);
```

```
      const response = await axiosClient.get('/articles');
      setArticles(response.data); // On met les articles recus dans notre state !
      setError(null);
    } catch (err) {
      setError('Serveur Express injoignable sur le port 5000.');
    } finally {
      setLoading(false);
    }
  };

  // On lance chargerArticles() une seule fois a l'ouverture du composant
  useEffect(() => {
    chargerArticles();
  }, []);

  if (loading) return <p>Chargement en cours...</p>;
  if (error) return <p style={{ color: 'red' }}>{error}</p>;

  return (
    <ul>
      {articles.map((a) => <li key={a.id}>{a.title} (par {a.author})</li>)}
    </ul>
  );
}
```

Point de contrôle. Observez bien le bloc `try / catch` ci-dessus : c'est exactement le même JavaScript que vous avez écrit en Séances 1 à 3 côté Express !

- Si le serveur répond → `response.data` remplit la liste.
- Si le serveur est éteint → le programme saute dans `catch` et affiche un message clair sans faire planter la page.

3 Étape 3 (Heure 2) : Pourquoi Ajouter Redux Toolkit ? Le Problème du Compteur de la `Navbar`

1. La Question qui Fait Tout Comprendre

Notre code avec `useState` fonctionne pour un seul composant. Mais sur notre écran DevBlog :

- En haut, la **`Navbar`** affiche le badge « **Articles en Base : 3** ».
- À gauche, **`ArticleForm`** ajoute un article (le badge doit passer à **4**).
- À droite, **`ArticleList`** affiche les cartes et permet d'en supprimer une (le badge repasse à **3**).

Si chaque composant garde son `useState` isolé, la `Navbar` et la liste ne se synchronisent pas sans passer des `props` partout (*Prop Drilling*). **La solution :** placer la liste dans un **Coffre-Fort Central** appelé le **Store Redux**.

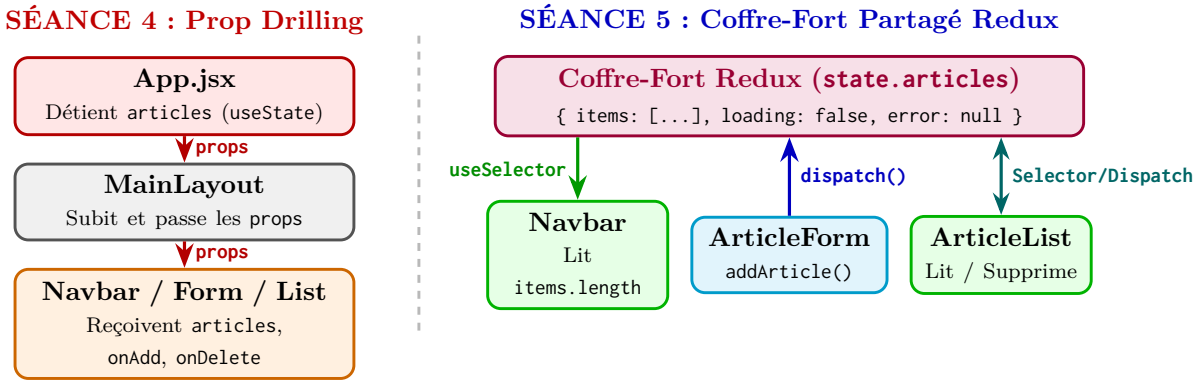


FIGURE 2 – Avec le Coffre-Fort Redux, chaque composant lit ou met à jour les articles directement sans `props`.

Repère Simple : Quand choisir `useContext` vs Redux Toolkit.

- **`useContext` (Natif React) :** Parfait pour un petit réglage global qui change rarement (Thème Clair/Sombre ou Langue).
- **Redux Toolkit (RTK) :** Idéal dès qu'on partage une **liste de données venue d'un serveur** (articles, panier) avec ajout, suppression et chargement.

2. Tout Redux Toolkit en 2 Commandes Seulement !

Une fois le coffre-fort créé, vos composants React n'utilisent que **2 petits Hooks** :

1. **Pour LIRE dans le coffre (`useSelector`) :** `const { items, loading, error } = useSelector((s) => s.articles);`
2. **Pour RANGER dans le coffre (`useDispatch`) :** `dispatch(setArticles(res.data));` ou `dispatch(addArticle(art));`



FIGURE 3 – La méthode la plus simple pour combiner Axios et Redux Toolkit en 3 temps A → B → C.

4 Étape 4 : TP Immédiat 2 – Créer le Coffre-Fort Redux en 25 Lignes

1. Installer Redux Toolkit dans le Dossier `client/`

```
npm install @reduxjs/toolkit react-redux
```

2. Créer le Tiroir des Articles (`src/redux/articleSlice.js`)

Regardez la simplicité de ce fichier : **aucune fonction compliquée** ! Il contient simplement l'état de départ (`initialState`) et 5 petites fonctions de 1 à 2 lignes qui remplacent les `setArticles`, `setLoading` et `setError` de tout à l'heure :

➥ Fichier `src/redux/articleSlice.js` – exécuter avec `npm run dev`

```
import { createSlice } from '@reduxjs/toolkit';

const articleSlice = createSlice({
  name: 'articles',
  initialState: {
    items: [],      // La liste des articles recus du serveur Express
    loading: false, // true pendant que la requete Axios est en cours
    error: null     // Message d'erreur si le serveur Express est eteint
  },
  reducers: {
    // 1. Remplir la liste quand Axios recoit les articles (GET)
    setArticles: (state, action) => {
      state.items = action.payload;
      state.loading = false;
      state.error = null;
    },

    // 2. Ajouter un nouvel article en tete de liste (POST)
    addArticle: (state, action) => {
      state.items.unshift(action.payload); // Ajout direct grace a Immer !
    },

    // 3. Supprimer un article par son identifiant (DELETE)
    removeArticle: (state, action) => {
      state.items = state.items.filter((a) => a.id !== action.payload);
    },

    // 4. Activer le chargement ou afficher une erreur
    setLoading: (state, action) => {
      state.loading = action.payload;
    },
    setError: (state, action) => {
      state.error = action.payload;
      state.loading = false;
    }
  }
});

// Export des fonctions pour nos composants
export const {
  setArticles, addArticle, removeArticle, setLoading, setError
} = articleSlice.actions;

export default articleSlice.reducer;
```

3. Créer le Coffre (`src/redux/store.js`) et l'Activer dans `src/main.jsx`

➥ Fichier `src/redux/store.js` – exécuter avec `npm run dev`

```
import { configureStore } from '@reduxjs/toolkit';
import articleReducer from './articleSlice';
```

```
export const store = configureStore({
  reducer: {
    articles: articleReducer // Nos articles sont ranges dans state.articles
  }
});
```

➥ Fichier `src/main.jsx` – exécuter avec `npm run dev`

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { Provider } from 'react-redux';
import { store } from './redux/store';
import App from './App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    {/* Le Provider ouvre l'acces au coffre Redux pour toute l'application */}
    <Provider store={store}>
      <App />
    </Provider>
  </React.StrictMode>
);
```

Point de contrôle. Comparez `articleSlice.js` avec le `useState` de la Séance 4 :

- `action.payload` est tout simplement le paramètre que vous passez entre parenthèses quand vous appelez `dispatch(setArticles(mesArticles))`.
- Dans `addArticle`, on écrit directement `state.items.unshift(action.payload)` car Redux Toolkit inclut **Immer**, qui crée la copie propre du tableau automatiquement !

5 Étape 5 : TP Immédiat 3 – Brancher nos 4 Composants React

Dans chaque composant, nous appliquons toujours la même recette en **2 lignes** :

1. **Axios** fait le travail réseau avec le serveur Express (`get`, `post` ou `delete`).
2. **`dispatch(...)`** met à jour le coffre Redux !

1. La Barre de Navigation avec Compteur (`src/components/Navbar.jsx`)

La `Navbar` lit `items.length` dans le coffre Redux pour afficher le badge et propose un bouton « Rafraîchir » :

➥ Fichier `src/components/Navbar.jsx` – exécuter avec `npm run dev`

```
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import axiosClient from '../api/axiosClient';
import { setArticles, setLoading, setError } from '../redux/articleSlice';

export default function Navbar() {
  // 1. On lit la liste des articles directement dans le coffre Redux
  const { items, loading } = useSelector((state) => state.articles);
  const dispatch = useDispatch();

  const handleRefresh = async () => {
    try {
      dispatch(setLoading(true));
      const response = await axiosClient.get('/articles');
      dispatch(setArticles(response.data));
    } catch (err) {
      dispatch(setError('Serveur Express injoignable sur http://localhost:5000.'));
    }
  };

  return (
    <nav className="navbar">
      <div className="nav-brand">
        <span className="logo-icon">*</span>
        <div>
          <h2>DevBlog Full-Stack <span className="badge-track">Decale S5</span></h2>
          <span className="nav-sub">React 18 + Axios + Redux Toolkit (createSlice)</span>
        </div>
      </div>

      <div className="nav-stats">
        <button className="btn-refresh" onClick={handleRefresh} disabled={loading}>
          {loading ? 'Chargement...' : 'Rafraichir'}
        </button>

        <div className="stat-pill">
          <span className="stat-label">Articles en Base :</span>
          <span className="stat-count">{items.length}</span>
        </div>
      </div>
    </nav>
  );
}
```

2. La Carte d'Article avec Suppression (`src/components/ArticleCard.jsx`)

Regardez la fonction `handleDelete` : **2 lignes** seulement ! On supprime sur Express avec Axios, puis on retire du coffre Redux avec `dispatch(removeArticle(article.id))` :

➥ Fichier `src/components/ArticleCard.jsx` – exécuter avec `npm run dev`

```
import React from 'react';
```

```
import { useDispatch } from 'react-redux';
import axiosClient from '../api/axiosClient';
import { removeArticle } from '../redux/articleSlice';

export default function ArticleCard({ article }) {
  const dispatch = useDispatch();

  const handleDelete = async () => {
    // 1. Supprimer sur le serveur Express via Axios
    await axiosClient.delete(`/articles/${article.id}`);
    // 2. Retirer l'article du Store Redux
    dispatch(removeArticle(article.id));
  };

  return (
    <article className="article-card">
      <div className="card-top">
        <span className="badge-category">{article.category}</span>
        <button className="btn-delete" onClick={handleDelete}>
          Supprimer
        </button>
      </div>

      <h3 className="article-title">{article.title}</h3>

      <div className="card-footer">
        <span className="author-info">Auteur : <strong>{article.author}</strong></span>
      </div>
    </article>
  );
}
```

3. Le Formulaire d'Ajout (`src/components/ArticleForm.jsx`)

Regardez la fonction `handleSubmit` :

1. `const response = await axiosClient.post('/articles', { title, author, category });`
2. `dispatch(addArticle(response.data));`

Et aussitôt, la carte apparaît à droite et le compteur de la `Navbar` augmente de 1 !

➥ Fichier `src/components/ArticleForm.jsx` – exécuter avec `npm run dev`

```jsx
import React, { useState } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import axiosClient from '../api/axiosClient';
import { addArticle } from '../redux/articleSlice';

const CATEGORIES = [
  'Architecture Web', 'Base de Donnees', 'Cybersecurite',
  'Intelligence Artificielle', 'DevOps & Cloud'
];

export default function ArticleForm() {
  const [title, setTitle] = useState('');
  const [author, setAuthor] = useState('');
  const [category, setCategory] = useState(CATEGORIES[0]);
  const [feedback, setFeedback] = useState('');

  const dispatch = useDispatch();
  const { loading } = useSelector((state) => state.articles);

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!title.trim() || !author.trim()) return;

    // 1. Envoyer l'article au serveur Express via Axios (POST)
    const response = await axiosClient.post('/articles', { title, author, category });

    // 2. Ranger l'article cree dans le Store Redux
    dispatch(addArticle(response.data));

    // 3. Vider les champs du formulaire
    setTitle('');
    setAuthor('');
    setFeedback('Article publie avec succes sur l API Express !');
  };

  return (
    <section className="form-card">
      <div className="form-header">
        <h3>Publier un Nouvel Article</h3>
        <p>Envoi avec axiosClient.post() puis rangement dans Redux.</p>
      </div>

      {feedback && <div className="feedback-alert success">{feedback}</div>}

      <form onSubmit={handleSubmit} className="article-form">
        <div className="form-group">
          <label>Titre de l'Article :</label>
          <input
            type="text"
            placeholder="Ex : Architecture Microservices"
            value={title}
            onChange={(e) => setTitle(e.target.value)}
          />
        </div>

        <div className="form-row">
          <div className="form-group">
```

```
            <label>Auteur :</label>
            <input
              type="text"
              placeholder="Ex : Dr. Abdelweheb GUEDDES"
              value={author}
              onChange={(e) => setAuthor(e.target.value)}
            />
          </div>

          <div className="form-group">
            <label>Categorie :</label>
            <select value={category} onChange={(e) => setCategory(e.target.value)}>
              {CATEGORIES.map((cat) => <option key={cat} value={cat}>{cat}</option>)}
            </select>
          </div>
        </div>

        <button type="submit" className="btn-submit" disabled={loading}>
          + Publier dans la Base Cloud
        </button>
      </form>
    </section>
  );
}
```

4. La Liste des Articles (`src/components/ArticleList.jsx`)

Regardez la fonction `chargerArticles` : c'est exactement la même fonction `try / catch` que nous avons vue à l'Étape 2, sauf qu'au lieu d'un state privé, elle range les articles dans le coffre Redux avec `dispatch(setArticles(response.data))` !

➥ Fichier `src/components/ArticleList.jsx` – exécuter avec `npm run dev`

```
import React, { useState, useEffect } from 'react';
import { useSelector, useDispatch } from 'react-redux';
import axiosClient from '../api/axiosClient';
import { setArticles, setLoading, setError } from '../redux/articleSlice';
import ArticleCard from './ArticleCard';

export default function ArticleList() {
  const dispatch = useDispatch();

  // 1. Etat Global (Redux) : liste des articles, chargement et erreur
  const { items, loading, error } = useSelector((state) => state.articles);

  // 2. Etat Local (useState) : categorie choisie dans le menu deroulant
  const [selectedCategory, setSelectedCategory] = useState('');

  // Fonction simple pour charger les articles avec Axios et les ranger dans Redux
  const chargerArticles = async () => {
    try {
      dispatch(setLoading(true));
      const response = await axiosClient.get('/articles');
      dispatch(setArticles(response.data));
    } catch (err) {
      dispatch(setError('Serveur Express injoignable sur http://localhost:5000.'));
    }
  };

  // Chargement initial a l'ouverture de la page
  useEffect(() => {
    chargerArticles();
  }, []);

  // Filtrage JavaScript simple
  const filteredItems = selectedCategory
    ? items.filter((a) => a.category === selectedCategory)
    : items;

  return (
    <section className="list-section">
      <div className="list-header">
        <div>
          <h3>Articles Synchronises avec l'API Express</h3>
          <p className="list-subtitle">Donnees recuperees depuis http://localhost:5000/api/
  articles</p>
        </div>

        <div className="filter-box">
          <label>Filtrer par domaine : </label>
          <select value={selectedCategory} onChange={(e) => setSelectedCategory(e.target.
  value)}>
            <option value="">Tous les domaines</option>
            <option value="Architecture Web">Architecture Web</option>
            <option value="Base de Donnees">Base de Donnees</option>
            <option value="Cybersecurite">Cybersecurite</option>
            <option value="Intelligence Artificielle">Intelligence Artificielle</option>
            <option value="DevOps & Cloud">DevOps & Cloud</option>
          </select>
        </div>
      </div>
```

```
      {/* CAS 1 : Bandeau d'alerte si le serveur Express est eteint */}
      {error && (
        <div className="error-banner">
          <div className="error-icon">!</div>
          <div className="error-text">
            <strong>Erreur de Communication avec l'API Express :</strong>
            <p>{error}</p>
          </div>
          <button className="btn-retry" onClick={chargerArticles}>
            Reessayer
          </button>
        </div>
      )}

      {/* CAS 2 : Spinner pendant le voyage de la requete HTTP */}
      {loading && (
        <div className="loading-box">
          <div className="spinner"></div>
          <p>Chargement des articles depuis l'API Express...</p>
        </div>
      )}

      {/* CAS 3 : Grille des articles recus */}
      {!loading && filteredItems.length > 0 && (
        <div className="articles-grid">
          {filteredItems.map((article) => (
            <ArticleCard key={article.id} article={article} />
          ))}
        </div>
      )}
    </section>
  );
}
```

6 Étape 6 : Résultat Visuel en Laboratoire & 3 Petits Défis Guidés

Vérifiez que votre application reproduit exactement ces trois comportements :

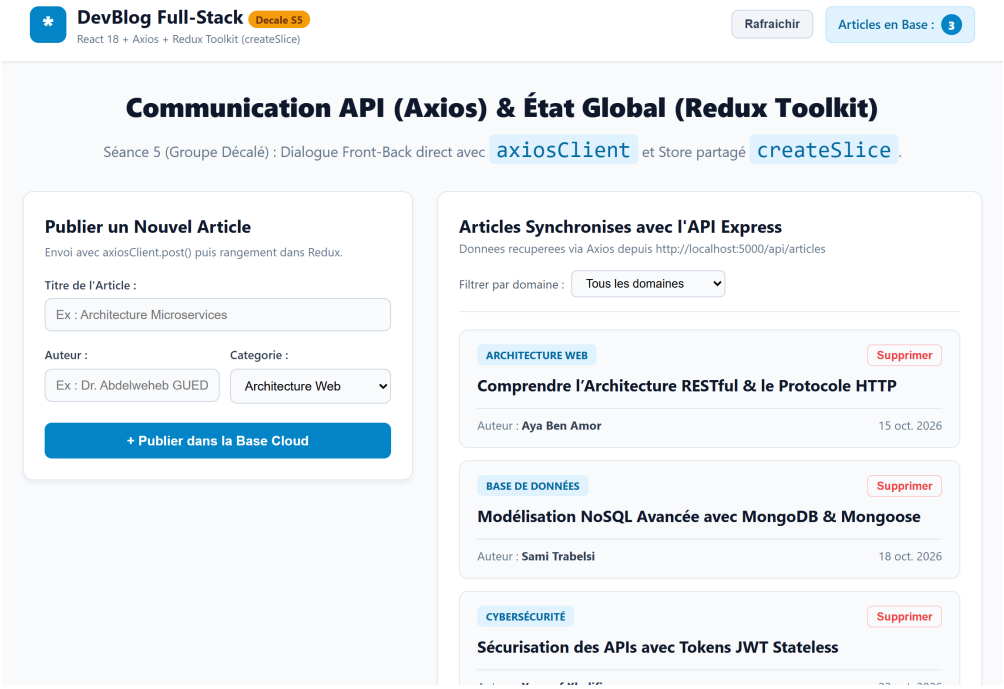
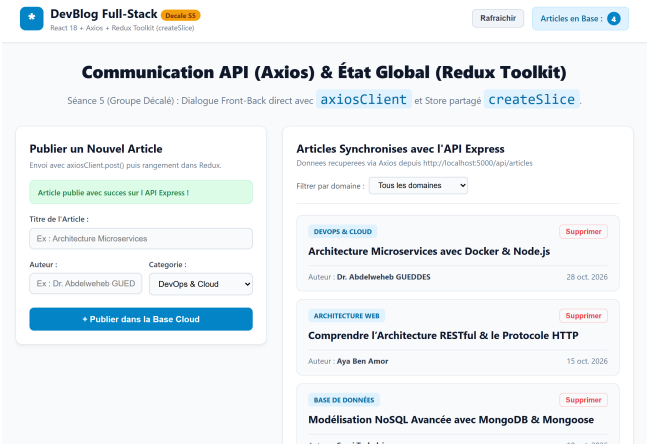
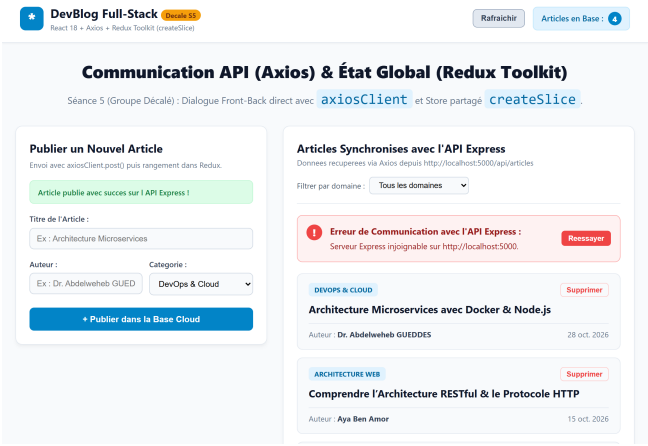


FIGURE 4 – **1. Chargement initial (`GET /api/articles`) :** Les 3 articles du serveur s'affichent et le badge en haut à droite indique `3`.



(a) **2. Publication (`POST`) + `dispatch(addArticle)` :** L'article s'ajoute en tête et le badge passe à `4`.



(b) **3. Serveur Express coupé (`catch`) :** L'alerte rouge s'affiche avec le bouton « Réessayer ».

FIGURE 5 – Fonctionnement en direct du couple Axios + Redux Toolkit (`createSlice`).

Les 3 Petits Défis Pratiques Guidés (Validation sur Écran)

Défi 1 : Ajouter une Catégorie & Afficher le Compteur Filtré ★

1. Dans `ArticleForm.jsx` et `ArticleList.jsx`, ajoutez la catégorie `"Genie Logiciel"`.
2. Dans `ArticleList.jsx`, juste sous le titre `<h3>`, affichez le nombre d'articles affichés :

```
<p className="list-subtitle">
  Affichage de <strong>{filteredItems.length}</strong> sur {items.length} articles.
</p>
```

Défi 2 : Barre de Recherche par Titre (`useState` + Redux) ★★

1. Dans `ArticleList.jsx`, ajoutez un state local : `const [search, setSearch] = useState('');`
2. Ajoutez un champ `<input type="text" placeholder="Rechercher un titre..." />` et filtrez :

```
const filteredItems = items.filter((a) => {
  const matchCat = selectedCategory === '' || a.category === selectedCategory;
  const matchSearch = a.title.toLowerCase().includes(search.toLowerCase());
  return matchCat && matchSearch;
});
```

Défi 3 : Demander Confirmation Avant de Supprimer ★★★

Dans `src/components/ArticleCard.jsx`, sécurisez le bouton « Supprimer » avec `window.confirm` :

```
const handleDelete = async () => {
  if (window.confirm(`Supprimer l'article "${article.title}" ?`)) {
    await axiosClient.delete(`/articles/${article.id}`);
    dispatch(removeArticle(article.id));
  }
};
```

Aide-Mémoire : Les 4 Erreurs Classiques & Leur Solution en 10 Secondes

Erreur Classique	Symptôme Observé	Solution Immédiate
1. Oubli de CORS	Erreur rouge dans la console : *"blocked by CORS policy"*.	Vérifier que `app.use(cors())` est bien présent dans `server/server.js`.
2. Oubli du `<Provider>`	Page blanche : *"could not find react-redux context value"*.	Entourer `<App />` avec `<Provider store={store}>` dans `main.jsx`.
3. Oubli de `.data`	`dispatch(setArticles(response))` plante la liste.	Toujours passer **`response.data`** : `dispatch(setArticles(response.data))`.
4. Serveur Express éteint	Le bandeau rouge affiche *"Serveur Express injoignable"*.	Ouvrir un terminal dans `server/` et lancer `npm start` (port `5000`).

Cap vers la Séance 6 Finale : Maintenant que vous savez appeler Express avec Axios et ranger les données dans Redux Toolkit, nous ajouterons en **Séance 6** la page de **Connexion JWT** et le **Déploiement Cloud** de votre projet !